

Junior .NET Cloud ERP Geliştirici El Kitabı

C# · Blazor Server · Entity Framework Core · SQL Server ile Kurumsal
Yazılım Geliştirmeye Başlangıç

Kişisel Başucu Kaynağı

Sürüm 1.0

İçindekiler

ÖNSÖZ	5
Bu Kitabı Nasıl Kullanmalı	5
Öğrenme Yol Haritası	6
BÖLÜM 1 — ERP TEMELLERİ	8
Tanım	8
Neden Önemli	9
MiniCloudERP’de Nerede Kullanılır	9
Kod Örneği	9
Sık Yapılan Hatalar	10
Özet	10
BÖLÜM 2 — BULUT BİLİŞİM (CLOUD COMPUTING)	11
Tanım	11
Neden Önemli	11
MiniCloudERP’de Nerede Kullanılır	12
Kod Örneği	12
Sık Yapılan Hatalar	12
Özet	13
BÖLÜM 3 — YAZILIM MİMARİSİNE GİRİŞ	14
Tanım	14
Neden Önemli	14
MiniCloudERP’de Nerede Kullanılır	14
Sık Yapılan Hatalar	15
Özet	15
BÖLÜM 4 — C# DİLİ	16
Tanım	16
Neden Önemli	16
MiniCloudERP’de Nerede Kullanılır	17
Kod Örneği	17
Sık Yapılan Hatalar	17

Özet	18
BÖLÜM 5 — NESNE YÖNELİMLİ PROGRAMLAMA VE SOLID	19
Tanım	19
Neden Önemli	20
MiniCloudERP’de Nerede Kullanılır	20
Kod Örneği	20
Sık Yapılan Hatalar	21
Özet	21
BÖLÜM 6 — ASP.NET CORE	22
Tanım	22
Neden Önemli	22
MiniCloudERP’de Nerede Kullanılır	22
Sık Yapılan Hatalar	23
Özet	24
BÖLÜM 7 — BLAZOR SERVER	25
Tanım	25
Neden Önemli	25
MiniCloudERP’de Nerede Kullanılır	26
Kod Örneği	26
Sık Yapılan Hatalar	27
Özet	27
BÖLÜM 8 — ENTITY FRAMEWORK CORE	28
Tanım	28
Neden Önemli	28
MiniCloudERP’de Nerede Kullanılır	29
Kod Örneği	29
Sık Yapılan Hatalar	30
Özet	30
BÖLÜM 9 — SQL SERVER	31
Tanım	31
Neden Önemli	31
MiniCloudERP’de Nerede Kullanılır	31
Sık Yapılan Hatalar	32
Özet	32
BÖLÜM 10 — IDENTITY VE GÜVENLİK	33
Tanım	33
Neden Önemli	33
MiniCloudERP’de Nerede Kullanılır	34
Kod Örneği	34
Sık Yapılan Hatalar	34
Özet	34
BÖLÜM 11 — KATMANLI MİMARİ (DETAYLI)	36

Tanım	36
Neden Önemli	36
MiniCloudERP’de Nerede Kullanılır	36
Kod Örneği	37
Sık Yapılan Hatalar	38
Özet	38
BÖLÜM 12 – GİT	39
Tanım	39
Neden Önemli	39
MiniCloudERP’de Nerede Kullanılır	39
Kod Örneği	40
Sık Yapılan Hatalar	40
Özet	40
BÖLÜM 13 – TEST YAZMA	41
Tanım	41
Neden Önemli	41
MiniCloudERP’de Nerede Kullanılır	41
Kod Örneği	42
Sık Yapılan Hatalar	42
Özet	43
BÖLÜM 14 – MINICLOUDERP’DE UÇTAN UCA AKIŞ	44
Senaryo: Bir Siparişin Baştan Sona Yolculuğu	44
Bu Akıştan Çıkarılacak Ders	45
EK A – HIZLI MÜLAKAT REFERANSI	46
ERP ve Bulut	46
C# ve OOP	46
ASP.NET Core ve Blazor	46
Entity Framework Core ve SQL Server	47
Identity, Mimari, Git ve Test	47
Genel Davranış Soruları	47
KAPANIŞ	48

JUNIOR .NET CLOUD ERP GELİŞTİRİCİ EL KİTABI

C# / .NET 8 / ASP.NET Core / Blazor Server / Entity Framework

Core / SQL Server

Bir Bilgisayar Mühendisliği mezununun, ERP üzerine Junior Yazılım Geliştirici olarak çalışırken kullanmak üzere hazırladığı kişisel başucu kaynağı

Sürüm 1.0

ÖNSÖZ

Bu kitap, ERP alanında çalışan bir junior yazılım geliştiricisinin kendi kullanımı için hazırladığı bir başucu kaynağıdır. Amacı bir üniversite dersi gibi baştan sona ezberletmek değil; gerçek iş hayatında bir kavramı unuttuğunda, bir mülakata hazırlanırken ya da bir kod incelemesinde karşılaştığı bir terimi hatırlamak istediğinde açıp bakabileceği pratik bir referans olmaktır.

Bilgisayar Mühendisliği eğitiminde programlamanın temelleri öğrenilir: değişkenler, döngüler, fonksiyonlar, nesne yönelimli programlama. Ancak bir okul projesi ile gerçek bir **Cloud ERP** ürünü arasında ciddi bir fark vardır.

Bir okul projesi genellikle:

- Tek bir kullanıcı tarafından çalıştırılır,
- Küçük ve kontrollü veri setleriyle test edilir,
- Bir kez teslim edilip bir daha dokunulmaz.

Bir Cloud ERP ürünü ise:

- Aynı anda onlarca, yüzlerce farklı şirket tarafından kullanılır,
- Yıllarca canlıda kalır ve bakım gerektirir,
- Başka geliştiricilerle birlikte, katmanlı bir mimari içinde geliştirilir,
- Para, stok, sipariş gibi gerçek iş verileriyle çalışır — bir hata gerçek bir müşteriye gerçek bir zarar verebilir.

Bu kitabın amacı, “kod yazabilen bir öğrenci” bakış açısından “kurumsal bir yazılımı anlayabilen ve sürdürebilen bir Junior .NET Developer” bakış açısına geçişi kolaylaştırmaktır.

Kitap boyunca kullanılan örnek proje, **MiniCloudERP** adında kurgusal ama gerçekçi bir Cloud ERP uygulamasıdır: müşteri yönetimi, ürün/stok yönetimi, sipariş ve fatura süreçlerini kapsayan; C# ve .NET 8 ile yazılmış, arayüzü Blazor Server, veritabanı SQL Server olan bir sistem.

Bu Kitabı Nasıl Kullanmalı

1. **İlk okumada sırayla ilerle.** Bölümler birbirinin üzerine inşa edilir; C# temelini atlayıp doğrudan Entity Framework’e geçmek, temel atmadan çatı kurmaya benzer.
2. **Kod örneklerini gerçekten çalıştır.** Sadece okumak yetmez — MiniCloudERP benzeri küçük bir proje açıp örneklerin gerçekten aynı şekilde çalıştığını gözlemlemek kalıcı öğrenme sağlar.
3. **“Unutma” kutularına dikkat et.** Bu kutular, mülakatlarda ve kod incelemelerinde en sık karşılaşılan noktaları vurgular.
4. **“MiniCloudERP’de” kutularına odaklan.** Soyut bir kavramı somut bir ERP senaryosuna bağlar; bir kavramı gerçek hayatta nerede kullanacağını gösterir.
5. **Ek A’yı (Hızlı Mülakat Referansı) mülakat öncesi tekrar tekrar oku.** Bu bölüm, kitabın geri kalanını tekrar etmeden hızlı hatırlama için tasarlanmıştır.

Unutma: Bu kitap bir ders kitabı değil, bir referans kitabıdır. Bir kavramı unuttuğunda tekrar açıp bakabileceğin bir “şirket el kitabı” gibi düşün.

Öğrenme Yol Haritası

AŞAMA 1 - TEMELLER
ERP Kavramları -> Bulut Bilişim -> Yazılım Mimarisine Giriş
|
v
AŞAMA 2 - DİL VE PARADİGMA
C# Dili -> Nesne Yönelimli Programlama
|
v
AŞAMA 3 - UYGULAMA KATMANI
ASP.NET Core -> Blazor Server
|
v
AŞAMA 4 - VERİ KATMANI
Entity Framework Core -> SQL Server
|
v
AŞAMA 5 - GÜVENLİK VE MİMARİ
Identity & Güvenlik -> Katmanlı Mimari
|
v
AŞAMA 6 - EKİP ÇALIŞMASI VE KALİTE
Git -> Test Yazma
|
v
AŞAMA 7 - BÜYÜK RESİM
ERP Projesinde Uçtan Uca Akış
|
v
AŞAMA 8 - MÜLAKATA HAZIRLIK
Hızlı Mülakat Referansı

Aşama	Tahmini Süre	Odak
1. Temeller	2-3 gün	Kavramsal ERP ve altyapı bilgisi
2. Dil ve Paradigma	1 hafta	C# ve OOP’de akıcılık
3. Uygulama Katmanı	1 hafta	ASP.NET Core + Blazor ile ekran/servis yazma
4. Veri Katmanı	4-5 gün	EF Core + SQL Server ile veri okuma/yazma
5. Güvenlik ve Mimari	3-4 gün	Identity + katmanlı mimari

Aşama	Tahmini Süre	Odak
6. Ekip Çalışması	2 gün	Git akışı ve test alışkanlığı
7. Büyük Resim	2 gün	Uçtan uca istek akışını anlama
8. Mülakat Hazırlığı	Sürekli	Tekrar ve pratik

BÖLÜM 1 — ERP TEMELLERİ

Tanım

ERP (Enterprise Resource Planning / Kurumsal Kaynak Planlama), bir şirketin temel iş süreçlerini — müşteri yönetimi, satış, stok, satın alma, muhasebe — **tek bir merkezi sistem** içinde birleştiren yazılım türüdür.

Cloud ERP, bu sistemin şirketin kendi sunucularında değil, Azure/AWS gibi bulut sağlayıcılarında çalıştığı; internet üzerinden herhangi bir cihazdan erişilebildiği modeldir.

Terim	Açıklama
On-Premise	Yazılımın şirketin kendi sunucularında kurulduğu klasik model
SaaS (Software as a Service)	Yazılımın abonelik modeliyle, kurulum gerektirmeden internet üzerinden sunulduğu model
CRM (Customer Relationship Management)	Müşteri ilişkilerini yöneten sistem; genellikle ERP'nin bir modülü veya ona entegre çalışan ayrı bir sistemdir
Business Module (İş Modülü)	ERP içindeki bağımsız ama birbirine bağlı iş birimi (örn. Stok Modülü, Muhasebe Modülü)

Temel İş Nesneleri (Business Entities)

Nesne	Ne İşe Yarar
Customer (Müşteri)	Ürün/hizmet satılan kişi veya şirket
Product (Ürün)	Satılan veya üretilen kalem
Category (Kategori)	Ürünlerin sınıflandırılması
Order (Sipariş)	Müşterinin verdiği sipariş
Stock (Stok)	Depodaki mevcut ürün miktarı
Warehouse (Depo)	Ürünlerin fiziksel olarak depolandığı yer
Invoice (Fatura)	Sipariş sonrası oluşturulan fatura belgesi
Supplier (Tedarikçi)	Şirkete ürün/hammadde sağlayan taraf
Procurement (Satın Alma)	Tedarikçiden ürün/hammadde satın alma süreci

Temel İş Süreçleri

- **Satış Süreci:** Müşteri sipariş verir → stok kontrol edilir → sipariş onaylanır → fatura kesilir → stok düşülür.
- **Satın Alma Süreci:** Stok azalır → tedarikçiye sipariş verilir → mal teslim alınır → stok artırılır → tedarikçi faturası işlenir.

- **Sipariş İş Akışı:** Created → Confirmed → Processing → Shipped → Completed (ya da Cancelled).
- **Stok İş Akışı:** Stock In (giriş) ve Stock Out (çıkış) hareketleriyle güncel stok seviyesi hesaplanır.

Neden Önemli

Kod yazmadan önce **neyin otomatikleştirildiğini** anlamak gerekir. ERP terminolojisi bilinmeden yazılan kod, teknik olarak doğru ama iş mantığı olarak yanlış olabilir. Örneğin “stok neden negatif olamaz” sorusunun cevabı bilinmezse doğru validasyon kodlanamaz.

ERP projelerinde ekipler genellikle iş analisti + geliştirici şeklinde çalışır; ancak junior geliştiricinin de temel iş terimlerini bilmesi, gereksinim dokümanlarını doğru okumasını ve gereksiz sorular sormadan geliştirme yapmasını sağlar.

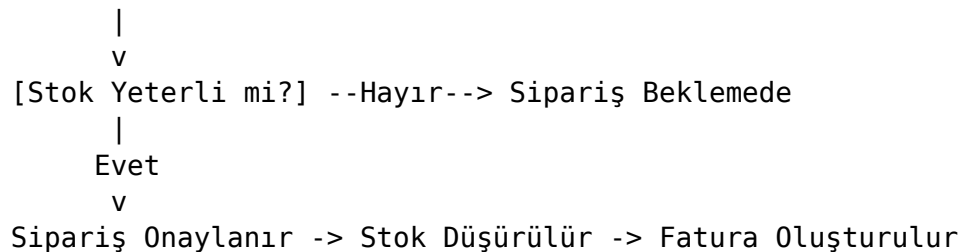
MiniCloudERP’de Nerede Kullanılır

Bu kavramlar, MiniCloudERP’nin veritabanı şemasının ve arayüz ekranlarının doğrudan temelini oluşturur: Customers, Products, Orders, Stocks, Invoices tabloları birebir bu iş nesnelerinin karşılığıdır.

Bir sipariş oluşturulduğunda sistem sırasıyla:

1. Customer kaydını doğrular (müşteri aktif mi?),
2. Order kaydı oluşturur (Status = Created),
3. Siparişteki her Product için Stock miktarını kontrol eder,
4. Yeterli stok varsa siparişi onaylar (Status = Confirmed) ve stoğu düşürür,
5. Invoice kaydı oluşturur.

Müşteri Siparişi



Kod Örneği

```

public class Order
{
    public int Id { get; set; }
    public int CustomerId { get; set; }
    public OrderStatus Status { get; set; } = OrderStatus.Created;
    public DateTime CreatedDate { get; set; } = DateTime.UtcNow;
    public List<OrderItem> Items { get; set; } = new();
}
  
```

```
public enum OrderStatus
{
    Created,
    Confirmed,
    Processing,
    Shipped,
    Completed,
    Cancelled
}
```

Sık Yapılan Hatalar

- ERP'yi sadece “bir CRUD uygulaması” gibi düşünmek; oysa iş süreçleri (workflow) ve durum geçişleri (state transitions) en kritik kısımdır.
- Stok kontrolü yapmadan siparişi onaylamak.
- “Cloud ERP” ile sıradan bir “web sitesi” kavramını karıştırmak; Cloud ERP çok kullanıcı, çok kiracılı (multi-tenant) ve yüksek veri bütünlüğü gerektiren bir sistemdir.

Unutma: ERP ile CRM arasındaki fark sık sorulur — ERP tüm şirket kaynaklarını (stok, muhasebe, üretim) yönetirken CRM sadece müşteri ilişkilerine odaklanır; CRM genellikle ERP'nin bir alt modülü ya da ona entegre ayrı bir sistemdir. SaaS'ta yazılım sağlayıcıda barınır ve aboneliği kullanılır; On-Premise'da yazılım müşterinin kendi sunucusuna kurulur ve bakımı müşteriye aittir.

Özet

ERP, bir şirketin tüm temel iş süreçlerini tek platformda birleştirir. Cloud ERP bunu internet üzerinden, ölçeklenebilir şekilde sunar. Customer, Product, Order, Stock, Invoice gibi temel nesneler ve bunların durum geçişleri (workflow), yazılan her kodun arkasındaki iş mantığını oluşturur.

BÖLÜM 2 – BULUT BİLİŞİM (CLOUD COMPUTING)

Tanım

Bulut (Cloud), sunucuları, depolamayı ve ağı kendi fiziksel donanımını almadan, internet üzerinden kiraladığınız modeldir. **Azure** (Microsoft) ve **AWS** (Amazon) en yaygın bulut sağlayıcılarıdır.

Terim	Açıklama
Server (Sunucu)	Uygulamanın çalıştığı, istekleri karşılayan bilgisayar/servis
Hosting	Uygulamanın bir sunucu üzerinde erişilebilir hale getirilmesi
Deployment	Kodun geliştirme ortamından canlı (production) ortama taşınması
Docker	Uygulamayı ve tüm bağımlılıklarını “container” adı verilen taşınabilir bir pakete koyan teknoloji
Container	Docker ile paketlenmiş, izole çalışan uygulama birimi
CI/CD	Continuous Integration / Continuous Deployment; kod değişikliklerinin otomatik test edilip otomatik yayına alınması süreci
Scaling (Ölçekleme)	Artan yükü karşılamak için kaynak (sunucu/kapasite) artırma
High Availability (Yüksek Erişilebilirlik)	Sistemin kesintisiz çalışmasını sağlayan mimari (birden fazla sunucu, otomatik yedekleme)
Caching (Önbellekleme)	Sık erişilen veriyi hızlı erişim için geçici bellekte tutma
Redis	Yaygın kullanılan, bellekte çalışan (in-memory) caching/veri deposu
Load Balancer	Gelen istekleri birden fazla sunucuya dağıtan bileşen
Reverse Proxy	İstemci isteklerini arka plandaki gerçek sunuculara yönlendiren ara katman (örn. NGINX)

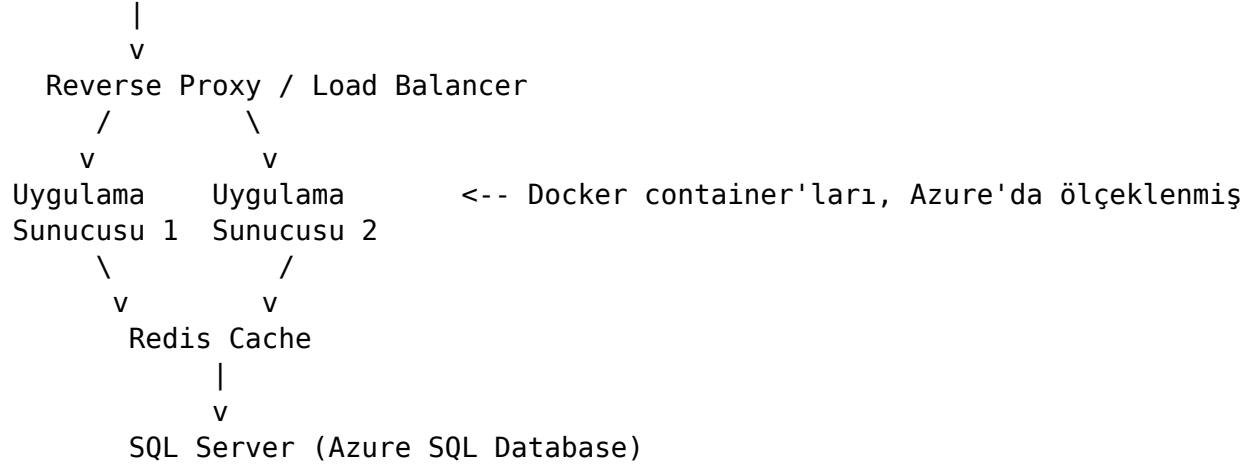
Neden Önemli

Bir Cloud ERP, tanımı gereği bulutta çalışır. Deployment, scaling ve caching kararları; uygulamanın yüzlerce şirket tarafından aynı anda, kesintisiz kullanılıp kullanılamayacağını belirler. Kodun tek bir sunucuda mı yoksa yük dengelenmiş birden fazla sunucuda mı çalışacağı bilinmelidir; çünkü bu, “session state” veya “in-memory cache” gibi konularda doğrudan tasarım kararlarını etkiler.

MiniCloudERP’de Nerede Kullanılır

- MiniCloudERP, Azure App Service üzerinde host edilir.
- Yoğun raporlama sorguları Redis ile önbelleklenir (örn. “bugünkü toplam satış” gibi değerler her istekte SQL Server’a gitmez).
- Yüksek trafik döneminde (örn. yıl sonu envanter sayımı) Load Balancer, gelen istekleri birden fazla uygulama sunucusuna dağıtır.
- CI/CD hattı, geliştiricinin gönderdiği her kod değişikliğini otomatik test edip onaylıysa otomatik yayına alır.

Kullanıcı Tarayıcısı



Kod Örneği

```
// Program.cs - Redis cache kaydı (kavramsal örnek)
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration = builder.Configuration["Redis:ConnectionString"];
});

// Bir servis içinde cache kullanımı
var cachedTotal = await _cache.GetStringAsync("daily-sales-total");
if (cachedTotal is null)
{
    var total = await _orderRepository.GetTodayTotalAsync();
    await _cache.SetStringAsync("daily-sales-total", total.ToString(),
        new DistributedCacheEntryOptions { AbsoluteExpirationRelativeToNow = TimeSpan.F
```

Sık Yapılan Hatalar

- Her veriyi önbelleklemeye çalışmak; sık değişen veriler (örn. anlık stok) için cache “eski veri gösterme” riski taşır.
- Tek sunucuya güvenip yüksek erişilebilirlik tasarlamamak; bir sunucu çöktüğünde tüm ERP’nin durmasına yol açar.

- Docker container'ı ile sanal makineyi (Virtual Machine) karıştırmak; container işletim sistemini paylaşır, sanal makine paylaşmaz — bu yüzden container çok daha hafif ve hızlı başlar.

Unutma: Load Balancer, gelen istekleri birden fazla sunucuya dağıtarak hem yükü dengeler hem de bir sunucu çöktüğünde diğer sunucuların isteklere cevap vermeye devam etmesini sağlar (yüksek erişilebilirlik). Cache kullanmanın en büyük riski, veritabanındaki veriyle önbellekteki verinin senkron kalmasını sağlamaktır — yanlış yönetilirse kullanıcıya güncel olmayan veri gösterilebilir.

Özet

Cloud ERP, sunucu/ağ/depolama kaynaklarını bulut üzerinden alarak ölçeklenebilirlik ve yüksek erişilebilirlik sağlar. Docker ve container'lar dağıtımı taşınabilir hale getirir, CI/CD teslimatı otomatikleştirir, Redis gibi önbellek çözümleri performansı artırır, Load Balancer ise yükü ve kesintisizliği yönetir.

BÖLÜM 3 – YAZILIM MİMARİSİNE GİRİŞ

Tanım

Yazılım Mimarisi, bir uygulamanın parçalarının (katmanlar, modüller, servisler) nasıl organize edildiğini ve birbiriyle nasıl iletişim kurduğunu tanımlayan üst düzey tasarımdır. Kurumsal yazılımlarda mimari, kodun büyüdükçe yönetilebilir kalmasını sağlayan iskelettir.

Terim	Açıklama
Monolith (Monolitik Mimari)	Tüm uygulamanın tek bir dağıtım birimi (tek proje/tek servis) olarak geliştirildiği mimari
Microservices (Mikroservis Mimarisi)	Uygulamanın bağımsız dağıtılabilen küçük servislere bölündüğü mimari
N-Tier / Layered Architecture (Katmanlı Mimari)	Uygulamanın sorumluluk bazında yatay katmanlara (Presentation, Business, Data) ayrıldığı mimari
Coupling (Bağımlılık)	İki bileşenin birbirine ne kadar sıkı bağlı olduğu
Cohesion (Uyum)	Bir bileşenin içindeki parçaların birbiriyle ne kadar ilişkili olduğu
API (Application Programming Interface)	Farklı yazılım parçalarının birbiriyle konuşmasını sağlayan sözleşme
Scalability (Ölçeklenebilirlik)	Sistemin artan yükü karşılayabilme kapasitesi

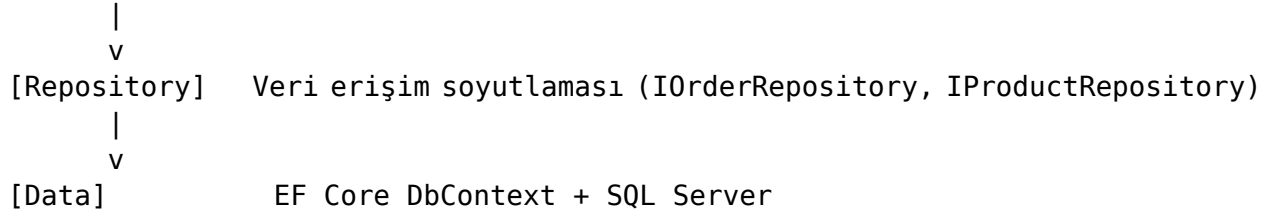
Neden Önemli

MiniCloudERP gibi bir ürün, tek bir geliştiricinin değil, zaman içinde değişen bir ekibin elinden geçer. Doğru mimari seçimi, yeni bir özelliğin eklenmesini kolaylaştırır; yanlış mimari ise her küçük değişikliği riskli ve yavaş hale getirir. Junior bir geliştirici olarak mimariyi *seçmek* zorunda kalınmaz genellikle, ama mevcut mimarinin kurallarına *saygı duymak* ve onu bozmadan geliştirme yapmak beklenir.

MiniCloudERP’de Nerede Kullanılır

MiniCloudERP, bir **monolitik, katmanlı mimari** ile geliştirilmiştir: tek bir ASP.NET Core çözümü içinde Presentation (Blazor), Business (Servisler), Repository ve Data (EF Core + SQL Server) katmanları bulunur. Bu, küçük-orta ölçekli bir ekip için mikroservislere göre daha az operasyonel karmaşıklık taşır; ürün büyüdükçe belirli modüller (örn. Faturalama) ayrı bir servise çıkarılabilir.

[Presentation] Blazor Server bileşenleri (kullanıcı arayüzü)
|
v
[Business] Servisler (OrderService, StockService – iş kuralları)



Sık Yapılan Hatalar

- Bir katmanı atlayıp doğrudan Blazor bileşeninden veritabanına sorgu yazmak; bu, katmanlı mimarinin tüm faydasını (test edilebilirlik, sorumluluk ayrımı) ortadan kaldırır.
- “Mikroservis daha modern” diye düşünüp küçük bir ekip için gereksiz operasyonel karmaşıklık (dağıtık sistem yönetimi) getirmek.
- Coupling (sıkı bağımlılık) ile Cohesion (uyum) kavramlarını karıştırmak: hedef düşük coupling, yüksek cohesion’dır.

Unutma: Monolitik mimari “kötü” değildir; küçük-orta ölçekli, tek ekip tarafından geliştirilen ürünler için genellikle mikroservislerden daha basit ve daha az maliyetlidir. Mikroservisler, ekip büyüdükçe ve modüller bağımsız ölçeklendirilmesi gerektiğinde anlamlı hale gelir.

Özet

Yazılım mimarisi, bir uygulamanın büyüme ve değişime ne kadar dayanıklı olacağını belirler. MiniCloudERP’nin katmanlı monolitik yapısı, bu kitabın ilerleyen bölümlerinde (ASP.NET Core, Blazor Server, EF Core, Katmanlı Mimari) somut olarak işlenecektir.

BÖLÜM 4 – C# DİLİ

Tanım

C#, Microsoft tarafından geliştirilen, statik tipli, nesne yönelimli bir programlama dilidir. .NET platformunun ana dili olarak ASP.NET Core, Blazor ve Entity Framework Core gibi teknolojilerin temelini oluşturur.

Terim	Açıklama
Class (Sınıf)	Nesnelerin şablonunu tanımlayan yapı; veri (property) ve davranışı (method) bir arada tutar
Interface (Arayüz)	Bir sınıfın “ne yapabildiğini” tanımlayan, uygulama detayını içermeyen sözleşme
Property	Bir sınıfın dışı açık verisi; genellikle get/set erişimcileriyle tanımlanır
Method (Metot)	Bir sınıfın davranışını tanımlayan fonksiyon
Constructor (Yapıcı Metot)	Bir nesne oluşturulduğunda çalışan, ilk kurulumu yapan özel metot
async / await	Asenkron (bloklamayan) kod yazmayı sağlayan anahtar kelimeler
Task / Task<T>	Asenkron bir işlemin sonucunu temsil eden tip; Task değer döndürmez, Task<T> bir T değeri döndürür
LINQ (Language Integrated Query)	Koleksiyonlar ve veritabanı sorguları üzerinde C# söz dizimiyle filtreleme/sıralama/gruplama yapmayı sağlayan özellik
Nullable Reference Types	Bir referans tipinin null olup olmayacağını derleme zamanında işaretleyen özellik (string? gibi)
record	Değer bazlı eşitlik sağlayan, genellikle değiştirilemez (immutable) veri taşımak için kullanılan tip
Generic (Jenerik)	Bir sınıf veya metodun, tipi parametre olarak alabilmesini sağlayan özellik (List<T> gibi)
Exception Handling	try/catch/finally blokları ile çalışma zamanı hatalarını yönetme mekanizması

Neden Önemli

C#, MiniCloudERP’nin tüm katmanlarında kullanılan ana dildir. Class/Interface yapısı Repository ve Service pattern’lerinin temelini oluşturur; async/await veritabanı ve ağ işlemlerinin performanslı çalışmasını sağlar; LINQ ise veri sorgulama işlemlerini okunabilir hale getirir.

MiniCloudERP’de Nerede Kullanılır

Bir sipariş servisinin veritabanından asenkron olarak sipariş çekmesi, LINQ ile filtrelemesi ve bir arayüz üzerinden dışa açılması, günlük geliştirme işinin büyük bölümünü oluşturur.

Kod Örneği

```
public interface IOrderRepository
{
    Task<List<Order>> GetOrdersByCustomerAsync(int customerId);
}

public class OrderRepository : IOrderRepository
{
    private readonly AppDbContext _context;

    public OrderRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task<List<Order>> GetOrdersByCustomerAsync(int customerId)
    {
        return await _context.Orders
            .Where(o => o.CustomerId == customerId && o.Status != OrderStatus.Cancelled)
            .OrderByDescending(o => o.CreatedDate)
            .ToListAsync();
    }
}
```

Bu örnekte Where ve OrderByDescending LINQ metotlarıdır; async/await ve Task<List<Order>> ise sorgunun veritabanı yanıtını beklerken thread’i bloklamadan çalışmasını sağlar.

Sık Yapılan Hatalar

- async bir metodu await etmeden çağırmak (.Result veya .Wait() kullanmak); bu, deadlock riskine ve beklenmeyen hatalara yol açabilir.
- LINQ sorgularını veritabanı tarafında değil, tüm veriyi belleğe çektikten sonra filtrelemek (örn. ToList() çağırıp sonra Where yazmak) — bu, gereksiz veri transferine ve performans kaybına yol açar.
- Nullable Reference Types uyarılarını (CS8602 gibi) görmezden gelmek; bu uyarılar genellikle gerçek bir NullReferenceException riskine işaret eder.

Unutma: Task geriye değer döndürmeyen asenkron işlemi, Task<T> ise T tipinde bir değer döndüren asenkron işlemi temsil eder. record, değer bazlı eşitlik sağlar ve genellikle değişmez veri taşımak için kullanılır; class

ise referans bazlı eşitlik kullanır ve genellikle değiştirilebilir durumu temsil eder. Interface, bir sınıfın “ne yapabildiğini” tanımlar ama “nasıl yaptığını” gizler — bu sayede farklı implementasyonlar birbirinin yerine kullanılabilir; bu, Dependency Injection’ın da temelidir.

Özet

C#, tip güvenliği, async/await ile performanslı asenkron kod yazımı ve LINQ ile okunabilir sorgular sunan, kurumsal yazılım geliştirmeye uygun olgun bir dildir. İleriki bölümlerdeki Repository, Service ve Dependency Injection kalıpları bu dilin özelliklerine dayanır.

BÖLÜM 5 — NESNE YÖNELİMLİ PROGRAMLAMA VE SOLID

Tanım

Nesne yönelimli programlama (OOP), yazılımı birbiriyle etkileşen nesneler olarak modelleyen paradigmadır. Kurumsal yazılımlarda kodun uzun vadede bakımı yapılabilir, genişletilebilir ve test edilebilir olması için belirli ilkeler benimsenir.

SOLID Prensipleri

Harf	İlke	Anlamı
S	Single Responsibility (Tek Sorumluluk)	Bir sınıfın sadece bir değişme sebebi olmalıdır
O	Open/Closed (Açık/Kapalı)	Sınıf genişletmeye açık, değiştirmeye kapalı olmalıdır
L	Liskov Substitution (Yerine Geçebilirlik)	Bir alt sınıf, üst sınıfın yerine sorunsuz kullanılabilir
I	Interface Segregation (Arayüz Ayrımı)	Kullanılmayan metotları zorunlu kılan büyük arayüzler yerine küçük, özel arayüzler tercih edilmelidir
D	Dependency Inversion (Bağımlılığın Tersine Çevrilmesi)	Üst seviye modüller alt seviye modüllere değil, soyutlamalara (interface) bağlanmalıdır

Diğer Önemli İlkeler

İlke	Anlamı
DRY (Don't Repeat Yourself)	Aynı mantığı birden fazla yerde tekrar etmemek
KISS (Keep It Simple, Stupid)	Gereksiz karmaşıklıktan kaçınmak
YAGNI (You Aren't Gonna Need It)	Şu an ihtiyaç olmayan özelliği önceden eklememek
Loose Coupling (Gevşek Bağımlılık)	Sınıfların birbirine mümkün olduğunca az bağımlı olması
High Cohesion (Yüksek Uyum)	Bir sınıf içindeki tüm üyelerin ortak, net bir amaca hizmet etmesi
Composition (Bileşim)	Bir nesneyi kalıtım yerine başka nesneleri içinde barındırarak oluşturmak ("has-a" ilişkisi)

Neden Önemli

MiniCloudERP gibi yıllarca bakımı yapılacak bir üründe, SOLID'e uyulmadan yazılan kod kısa sürede "spagetti koda" dönüşür. Özellikle **Dependency Inversion**, Repository Pattern ve Dependency Injection'ın temelini oluşturur: OrderService, IOrderRepository'ye bağımlıdır, somut OrderRepository sınıfına değil. Bu sayede repository'nin iç implementasyonu (SQL Server, başka bir veritabanı, test için sahte veri) değiştiğinde OrderService hiç etkilenmez.

MiniCloudERP'de Nerede Kullanılır

- **Single Responsibility:** OrderService sadece sipariş iş mantığından, InvoiceService sadece fatura oluşturmaktan sorumludur.
- **Dependency Inversion:** Tüm servisler somut repository yerine IRepository arayüzlerine bağımlıdır.
- **Open/Closed:** Yeni bir ödeme yöntemi eklenirken mevcut IPaymentProcessor implementasyonlarına dokunulmadan yeni bir implementasyon eklenir.

Kötü Tasarım (SRP ihlali):

OrderService

- Sipariş oluşturur
- Fatura PDF'i oluşturur
- E-posta gönderir
- Stok günceller

(Tek sınıf, dört ayrı sorumluluk -> değişiklik riski yüksek)

İyi Tasarım (SRP'ye uygun):

OrderService

-> sipariş oluşturma iş mantığı

InvoiceService

-> fatura oluşturma

NotificationService

-> e-posta/bildirim gönderme

StockService

-> stok güncelleme

Kod Örneği

```
// Dependency Inversion + Interface Segregation
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(int id);
    Task AddAsync(Order order);
}

public class OrderService
{
    private readonly IOrderRepository _orderRepository;
    private readonly IStockService _stockService;

    // Somut sınıflara değil, arayüzlere bağımlı (Dependency Inversion)
    public OrderService(IOrderRepository orderRepository, IStockService stockService)
```

```

{
    _orderRepository = orderRepository;
    _stockService = stockService;
}

public async Task<Order> CreateOrderAsync(Order order)
{
    await _stockService.ReserveStockAsync(order.Items);
    await _orderRepository.AddAsync(order);
    return order;
}
}

```

Sık Yapılan Hatalar

- Bir sınıfa “yardımcı” (Helper/Manager) adı verip içine ilgisiz onlarca metod yığmak; bu Single Responsibility ve High Cohesion ihlalidir.
- YAGNI’yi göz ardı edip “ileride lazım olur” diye kullanılmayan genel (generic) yapılar kurmak; bu genellikle kodu gereksiz karmaşılaştırır.
- Kalıtımı (inheritance) her yerde composition yerine kullanmak; “is-a” ilişkisi olmayan yerlerde kalıtım kullanmak Liskov Substitution ihlaline yol açar.

Unutma: SOLID’deki “D” (Dependency Inversion), üst seviye iş mantığı kodunun alt seviye detaylara (veritabanı, dış servis) değil soyutlamalara bağlı olmasını sağlar; böylece detaylar değiştiğinde iş mantığı etkilenmez ve test edilebilirlik artar. DRY ile YAGNI birbiriyle çelişmez: DRY tekrar eden kodu birleştirmeyi, YAGNI ise henüz ihtiyaç olmayan özelliği eklememeyi savunur — ikisi birlikte “gerekeni, tekrarsız yaz” ilkesini oluşturur.

Özet

SOLID ve tamamlayıcı ilkeler (DRY, KISS, YAGNI), MiniCloudERP gibi uzun ömürlü bir ürünün sürdürülebilir olmasını sağlar. Bu ilkeler, Repository Pattern, Service Pattern ve Dependency Injection’ın “neden bu şekilde tasarlandığını” açıklayan temel gerekçedir.

BÖLÜM 6 — ASP.NET CORE

Tanım

ASP.NET Core, .NET üzerinde web uygulamaları ve API'ler geliştirmek için kullanılan, platform bağımsız (Windows/Linux/macOS) bir web framework'üdür.

Terim	Açıklama
Web Application	Tarayıcıda görüntülenen, HTML üreten uygulama türü
Web API	Veri (genellikle JSON) döndüren, arayüz içermeyen uygulama türü
MVC (Model-View-Controller)	Veri (Model), görüntü (View) ve istek işleme (Controller) sorumluluklarını ayıran mimari desen
Minimal API	Controller sınıfı olmadan, doğrudan Program.cs içinde endpoint tanımlama yaklaşımı
Kestrel	ASP.NET Core'un varsayılan, platform bağımsız web sunucusu
Dependency Injection (DI)	Bir sınıfın ihtiyaç duyduğu bağımlılıkların dışarıdan sağlandığı tasarım yaklaşımı
Routing	Gelen URL'nin hangi kod bloğuna (endpoint) yönlendirileceğini belirleyen mekanizma
Authentication (Kimlik Doğrulama)	Kullanıcının kim olduğunu doğrulama süreci
Authorization (Yetkilendirme)	Doğrulan kullanıcı belirlenen bir işlemi yapmaya yetkisi olup olmadığını kontrol etme
Options Pattern	Konfigürasyon değerlerini (appsettings.json) güçlü tipli C# sınıflarına bağlama yaklaşımı

Neden Önemli

Dependency Injection, MiniCloudERP'nin bütün katmanlarını birbirine gevşek bağlı (loosely coupled) şekilde bağlar. Program.cs içinde bir servis "container"a kaydedilir; ihtiyaç duyan her sınıf bu servisi constructor'ı aracılığıyla otomatik alır. Bu, hem test edilebilirliği hem de bakım kolaylığını sağlar.

MiniCloudERP'de Nerede Kullanılır

MiniCloudERP'de tüm Repository ve Service sınıfları Program.cs içinde DI container'ına kaydedilir:

```
builder.Services.AddScoped<IOrderRepository, OrderRepository>();
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<IStockService, StockService>();
```

Böylece bir Blazor bileşeni veya API controller'ı OrderService'e ihtiyaç duyduğunda onu new ile oluşturmaz; DI container otomatik sağlar:

DI Container'a Kayıt (Program.cs)

```
|
v
Kullanıcı "Siparişlerim" sayfasını açar
|
v
OrdersPage.razor -- constructor'da ister --> IOrderService
|
v
DI Container -- otomatik sağlar --> OrderService örneği
|
v
OrderService -- constructor'da ister --> IOrderRepository
```

Servis Ömürleri (Service Lifetimes)

Lifetime	Anlamı	Ne Zaman Kullanılır
AddTransient	Her istekte yeni bir örnek oluşturulur	Durum tutmayan, hafif servisler
AddScoped	Aynı HTTP isteği (veya Blazor devresi) içinde tek bir örnek paylaşılır	DbContext, Repository, Service sınıfları — ERP'de en sık kullanılan tür
AddSingleton	Uygulama boyunca tek bir örnek paylaşılır	Konfigürasyon, cache gibi durum tutan ama kullanıcıya özel olmayan servisler

Sık Yapılan Hatalar

- DbContext gibi bağlantıya duyarlı bir servisi Singleton olarak kaydetmek; bu, farklı kullanıcı istekleri arasında veri karışmasına ve eşzamanlılık hatalarına yol açar — DbContext neredeyse her zaman Scoped olmalıdır.
- Authentication (kimlik doğrulama) ile Authorization'ı (yetkilendirme) karıştırmak: biri "kimsin", diğeri "bunu yapmaya yetkin var mı" sorusuna cevap verir.
- Routing kurallarını düzensiz bırakıp aynı URL desenine birden fazla endpoint'in cevap vermesine izin vermek.

Unutma: AddScoped, aynı HTTP isteği veya Blazor Server devresi (circuit) içinde tek bir örnek paylaşılmasını sağlar — bu, DbContext için doğru seçimdir çünkü DbContext thread-safe değildir ve istek bazında izole

edilmelidir. AddSingleton kullanırken dikkat edilmesi gereken risk, o servisin kullanıcılar arasında paylaşılan durum (state) tutmasıdır; kullanıcıya özel veri asla Singleton bir serviste tutulmamalıdır.

Özet

ASP.NET Core, Dependency Injection, Routing ve Authentication/Authorization altyapısıyla MiniCloudERP'nin uygulama katmanının omurgasını oluşturur. Servis ömürlerinin (Transient/Scoped/Singleton) doğru seçilmesi, hem doğruluk hem de performans açısından kritik bir karardır.

BÖLÜM 7 — BLAZOR SERVER

Tanım

Blazor Server, C# ile etkileşimli web arayüzleri geliştirmeyi sağlayan bir ASP.NET Core teknolojisidir. JavaScript yazmadan, C# ve Razor söz dizimiyle bileşen (component) tabanlı arayüzler oluşturulur. Kullanıcı etkileşimleri **SignalR** üzerinden gerçek zamanlı olarak sunucuya iletilir; işlenen sonuç yalnızca değişen HTML parçası olarak tarayıcıya geri gönderilir.

Terim	Açıklama
Component (Bileşen)	Kendi arayüzünü ve mantığını içeren, tekrar kullanılabilir .razor dosyası
SignalR	Sunucu ile tarayıcı arasında gerçek zamanlı, kalıcı bağlantı sağlayan teknoloji; Blazor Server'ın temelidir
Circuit (Devre)	Bir kullanıcının tarayıcı oturumu ile sunucu arasındaki, bileşen durumunu (state) tutan canlı bağlantı
Render Tree	Bir bileşenin o anki arayüz çıktısının bellekte tutulan temsili
Two-Way Binding (@bind)	Bir arayüz elemanı ile C# değişkeni arasında değerlerin karşılıklı senkronize edilmesi
Lifecycle Method	Bir bileşenin yaşam döngüsü boyunca (oluşturma, veri yükleme, güncelleme) çağrılan özel metotlar (OnInitializedAsync gibi)
StateHasChanged	Bileşenin arayüzünü manuel olarak yeniden render etmesi gerektiğini bildiren metot
Cascading Parameter	Bir değeri, bileşen ağacındaki tüm alt bileşenlere otomatik aktarma mekanizması

Neden Önemli

Blazor Server'da her kullanıcı için sunucuda açık bir SignalR bağlantısı (circuit) ve bileşen durumu tutulur. Bu, geliştirmeyi C# ile tek dilde birleştirirken, sunucu tarafında dikkat edilmesi gereken yeni bir maliyet getirir: çok sayıda eşzamanlı kullanıcı, sunucu belleğinde çok sayıda açık circuit demektir. Bu nedenle Blazor Server'da performans ve bellek yönetimi, klasik bir web uygulamasından farklı düşünülmelidir.

MiniCloudERP’de Nerede Kullanılır

“Siparişlerim” sayfası açıldığında `OnInitializedAsync` içinde `IOrderService` üzerinden veriler yüklenir; kullanıcı bir siparişi onayladığında buton tıklaması SignalR üzerinden sunucuya iletilir, ilgili C# metodu çalışır ve yalnızca değişen tablo satırı tarayıcıya geri gönderilir — sayfa tam olarak yenilenmez.

Kullanıcı butona tıklar

|

v

SignalR bağlantısı üzerinden sunucudaki
bileşen örneğine olay iletilir

|

v

İlgili C# metodu (örn. `OnConfirmOrderClicked`) çalışır

|

v

Bileşenin yeni render çıktısı hesaplanır

|

v

Sadece değişen HTML parçası tarayıcıya gönderilir

Kod Örneği

```
@page "/orders"
@Inject IOrderService OrderService

<h3>Siparişlerim</h3>

@if (orders is null)
{
    <p>Yükleniyor...</p>
}
else
{
    <table class="table">
        @foreach (var order in orders)
        {
            <tr>
                <td>@order.Id</td>
                <td>@order.Status</td>
                <td>
                    <button onclick="() => ConfirmOrderAsync(order.Id)">Onayla</button>
                </td>
            </tr>
        }
    </table>
}
```

```
@code {
    private List<Order>? orders;

    protected override async Task OnInitializedAsync()
    {
        orders = await OrderService.GetOrdersForCurrentUserAsync();
    }

    private async Task ConfirmOrderAsync(int orderId)
    {
        await OrderService.ConfirmOrderAsync(orderId);
        orders = await OrderService.GetOrdersForCurrentUserAsync();
        StateHasChanged();
    }
}
```

Sık Yapılan Hatalar

- Bir bileşen içinde uzun süren, bloklayıcı bir işlem çalıştırmak; bu, o an için tüm SignalR bağlantısını (ve dolayısıyla o kullanıcının arayüzünü) yavaşlatır.
- OnInitializedAsync yerine constructor benzeri senkron kod içinde veri yüklemeye çalışmak; asenkron yükleme desenini doğru kullanmamak.
- SignalR bağlantısının kopabileceğini (örn. zayıf internet, sunucu yeniden başlatma) hiç göz önünde bulundurmadan tasarım yapmak — kullanıcı deneyimi bu durumlarda bozulabilir.

Unutma: Blazor Server’da bir tıklama olayı SignalR üzerinden sunucudaki bileşen örneğine iletilir, ilgili C# metodu çalışır, bileşenin yeni render çıktısı hesaplanır ve sadece değişen HTML parçası tarayıcıya geri gönderilir. Bu süreç tam sayfa yenilenmesi olmadan akıcı bir kullanıcı deneyimi sağlar — ama sunucu tarafında ağır/bloklayıcı bir işlem çalıştırmak o kullanıcının tüm arayüzünü dondurabilir.

Özet

Blazor Server, C# ile etkileşimli arayüzler geliştirmeyi mümkün kılar; SignalR üzerinden çalışan bu model hızlı geliştirme sağlarken sunucu tarafında bellek ve bağlantı yönetimini önemli bir konu haline getirir. MiniCloudERP’nin tüm kullanıcı arayüzü bu teknoloji üzerine kuruludur.

BÖLÜM 8 — ENTITY FRAMEWORK CORE

Tanım

Entity Framework Core (EF Core), C# nesnelerini veritabanı tablolarına eşleyen (Object-Relational Mapping / ORM) bir kütüphanedir. SQL sorguları elle yazmak yerine, LINQ ile C# nesneleri üzerinden veritabanı işlemleri yapılmasını sağlar.

Terim	Açıklama
DbContext	Veritabanı ile uygulama arasındaki oturumu ve tabloları temsil eden ana sınıf
DbSet<T>	DbContext içinde bir tabloyu temsil eden koleksiyon (örn. DbSet<Order> Orders)
Migration	Veritabanı şemasındaki değişiklikleri (yeni tablo, yeni kolon) sürüm kontrolüne benzer şekilde takip eden mekanizma
Fluent API	Entity ilişkilerini ve kısıtlamalarını C# kodu ile (attribute yerine) yapılandırma yaklaşımı
Change Tracker	EF Core'un, bellekteki nesnelerde yapılan değişiklikleri takip eden mekanizması
SaveChangesAsync	Change Tracker'ın izlediği değişiklikleri veritabanına asenkron olarak yazan metot
Eager Loading (Include)	İlişkili verileri ana sorguyla birlikte, tek seferde yükleme
Lazy Loading	İlişkili verinin yalnızca gerçekten erişildiği anda otomatik yüklenmesi
N+1 Problemi	Bir liste sorgusunun ardından her öge için ayrı ayrı ek sorgu atılması sonucu oluşan performans sorunu
Transaction	Birden fazla veritabanı işleminin ya tamamen başarılı ya da tamamen geri alınmış olmasını garanti eden mekanizma

Neden Önemli

EF Core, MiniCloudERP'nin veri katmanının merkezindedir. Migration'lar sayesinde veritabanı şeması kod ile birlikte versiyonlanır; Change Tracker sayesinde hangi entity'lerin değiştiği elle takip edilmez. Ancak EF Core'un rahatlığı yanlış kullanıldığında ciddi performans sorunlarına (özellikle N+1 problemi) yol açabilir.

MiniCloudERP’de Nerede Kullanılır

Bir sipariş, ilişkili OrderItem kayıtlarıyla birlikte tek sorguda çekilirken Include kullanılır; bir sipariş onaylanıp stok düşürülürken bu iki işlem bir Transaction içinde birlikte yürütülür.

Kod Örneği

```
public class AppDbContext : DbContext
{
    public DbSet<Order> Orders => Set<Order>();
    public DbSet<OrderItem> OrderItems => Set<OrderItem>();
    public DbSet<Product> Products => Set<Product>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Order>()
            .HasMany(o => o.Items)
            .WithOne()
            .HasForeignKey(i => i.OrderId);
    }
}

// Eager Loading ile N+1 problemini önleme
var orders = await _context.Orders
    .Include(o => o.Items)
    .Where(o => o.CustomerId == customerId)
    .ToListAsync();

// Transaction ile birden fazla tabloyu tutarlı güncelleme
using var transaction = await _context.Database.BeginTransactionAsync();
try
{
    _context.Orders.Add(order);
    product.StockQuantity -= orderedQuantity;
    await _context.SaveChangesAsync();
    await transaction.CommitAsync();
}
catch
{
    await transaction.RollbackAsync();
    throw;
}
```

Sık Yapılan Hatalar

- Bir sipariş listesi çekildikten sonra her sipariş için ayrı ayrı `context.Items.Where(...)` çağırarak; bu klasik **N+1 problemidir** — 100 sipariş için 1 ana sorgu yerine 101 sorgu çalışır. Çözüm: `Include` ile ilişkili veriyi baştan yüklemek.
- Stok düşürme ve sipariş kaydını ayrı ayrı, `Transaction` olmadan `SaveChangesAsync` çağrılarını yapmak; aradan bir hata geçerse veri tutarsızlığı (stok azalmış ama sipariş kaydedilmemiş) oluşur.
- Migration'ları production veritabanına test etmeden, doğrudan uygulamak; bu, beklenmeyen veri kaybına yol açabilir.

Unutma: N+1 problemi, bir liste sorgusunun ardından her öge için ayrı sorgu çalıştırılması sonucu oluşur ve `Include (eager loading)` ile önlenir. `Transaction`, birden fazla tabloyu etkileyen bir işlemin (örn. sipariş kaydı + stok güncelleme) ya tamamen başarılı ya da hiç uygulanmamış olmasını garanti eder — bu olmadan yapılan çok adımlı işlemler veri tutarsızlığı riski taşır.

Özet

EF Core, veritabanı işlemlerini C# nesneleri üzerinden yönetmeyi sağlar. Migration'lar şema değişikliklerini kod ile senkronize tutar, `Include/Transaction` gibi araçlar doğru ve performanslı veri erişimi için kritik öneme sahiptir.

BÖLÜM 9 — SQL SERVER

Tanım

SQL Server, Microsoft'un ilişkisel veritabanı yönetim sistemidir (RDBMS). Veriler tablolar halinde, satır ve sütunlarla, birbiriyle ilişkili şekilde saklanır.

Terim	Açıklama
Primary Key (Birincil Anahtar)	Bir tablodaki her satırı benzersiz şekilde tanımlayan alan
Foreign Key (Yabancı Anahtar)	Bir tablodaki bir alanın, başka bir tablonun birincil anahtarına referans vermesi
Index (Dizin)	Sorgu performansını artırmak için belirli kolonlar üzerinde oluşturulan hızlı arama yapısı
Normalization (Normalizasyon)	Veri tekrarını azaltmak için tabloları mantıksal olarak bölme süreci
JOIN	Birden fazla tablodaki veriyi ortak bir alan üzerinden birleştirerek sorgulama
Stored Procedure	Veritabanında önceden tanımlanmış, tekrar çalıştırılabilir SQL kod bloğu
ACID	Atomicity, Consistency, Isolation, Durability — bir veritabanı işleminin güvenilirliğini tanımlayan dört özellik
Deadlock	İki işlemin birbirinin serbest bırakmasını beklediği kilitlerin bir döngü oluşturması sonucu ikisinin de ilerleyemediği durum
Execution Plan	SQL Server'ın bir sorguyu nasıl çalıştıracakını gösteren, performans analizinde kullanılan plan

Neden Önemli

MiniCloudERP'nin tüm verisi SQL Server'da saklanır. Doğru index'ler olmadan, veri arttıkça sorgular yavaşlar; yanlış normalizasyon veri tutarsızlığına yol açar. Bir Junior Developer'ın SQL Server'ı EF Core "arkasında" çalışan bir kara kutu değil, performans ve doğruluk için doğrudan anlaşılması gereken bir bileşen olarak görmesi gerekir.

MiniCloudERP'de Nerede Kullanılır

Orders tablosunda CustomerId üzerinde bir index, müşteri bazlı sipariş sorgularını hızlandırır. OrderItems tablosu, OrderId ve ProductId üzerinden Orders ve Products tablolarına foreign key ile bağlıdır — bu, normalizasyonun temel bir örneğidir: ürün bilgisi her sipariş satırında tekrar edilmez, sadece referans edilir.

```
-- CustomerId üzerinde index: sipariş sorgularını hızlandırır
CREATE INDEX IX_Orders_CustomerId ON Orders(CustomerId);

-- Bir müşterinin siparişlerini ürün adlarıyla birlikte getirme
SELECT o.Id, o.Status, p.Name AS ProductName, oi.Quantity
FROM Orders o
JOIN OrderItems oi ON oi.OrderId = o.Id
JOIN Products p ON p.Id = oi.ProductId
WHERE o.CustomerId = @CustomerId;
```

Sık Yapılan Hatalar

- Sık sorgulanan kolonlarda (örn. CustomerId, Status) index olmaması; veri büyüdükçe sorgular saniyeler sürebilir.
- Aşırı normalizasyon yaparak basit bir raporlama sorgusunu onlarca JOIN gerektiren bir hale getirmek — bazı durumlarda kontrollü tekrar (denormalizasyon) performans için tercih edilebilir.
- Yoğun trafikli tablolarda uzun süren işlemleri aynı satırlar üzerinde kilit tutarak çalıştırmak; bu, Deadlock riskini artırır.

Unutma: ACID (Atomicity, Consistency, Isolation, Durability), bir veritabanı işleminin güvenilir sayılması için sağlaması gereken dört özelliktir — özellikle “Atomicity” (ya hep ya hiç) ve “Isolation” (eşzamanlı işlemlerin birbirini bozmaması), ERP gibi finansal veriyle çalışan sistemlerde kritik önem taşır. Index, sorgu hızını artırır ama her INSERT/UPDATE işleminde ek bir güncelleme maliyeti getirir — bu yüzden “her kolona index koy” yaklaşımı yanlıştır.

Özet

SQL Server, MiniCloudERP’nin verisinin güvenilir ve tutarlı şekilde saklandığı katmandır. Doğru index’leme, normalizasyon dengesi ve ACID prensiplerinin anlaşılması, EF Core üzerinden yazılan kodun gerçek performansını ve doğruluğunu belirler.

BÖLÜM 10 — IDENTITY VE GÜVENLİK

Tanım

ASP.NET Core Identity, kullanıcı yönetimi (kayıt, giriş, şifre sıfırlama), rol yönetimi ve kimlik doğrulama işlemlerini hazır olarak sağlayan bir kütüphanedir. Güvenlik, bir ERP sisteminde para ve iş verisine erişimi kontrol ettiği için mimarının en kritik parçalarından biridir.

Terim	Açıklama
Authentication (Kimlik Doğrulama)	“Bu kullanıcı gerçekten kim olduğunu iddia ediyor mu” sorusuna cevap veren süreç
Authorization (Yetkilendirme)	“Bu kullanıcı bu işlemi yapmaya yetkili mi” sorusuna cevap veren süreç
Role-Based Authorization	Kullanıcının rolüne göre (Admin, User gibi) yetki verilmesi
Claim-Based Authorization	Kullanıcı hakkındaki ince taneli bilgilere (claim) göre yetki verilmesi (örn. “Department: Sales”)
Policy (Politika)	Bir veya birden fazla claim/rol kuralını birleştiren, yeniden kullanılabilir yetkilendirme kuralı
JWT (JSON Web Token)	Kullanıcı kimliğini ve yetkilerini imzalı bir token içinde taşıyan, API’ler arası kimlik doğrulamada yaygın kullanılan format
Hashing (Özetleme)	Bir şifrenin geri döndürülemez şekilde şifrelenmiş halini üretme; şifreler asla düz metin olarak saklanmaz
HTTPS/TLS	İstemci ile sunucu arasındaki veri trafiğini şifreleyen protokol
SQL Injection	Kullanıcı girdisinin doğrudan SQL sorgusuna eklenmesi sonucu oluşan güvenlik açığı
XSS (Cross-Site Scripting)	Kötü amaçlı script’in bir sayfaya enjekte edilerek başka kullanıcılarda çalıştırılması açığı

Neden Önemli

Bir ERP sisteminde, hangi kullanıcının hangi müşterinin siparişini görebileceği, hangi kullanıcının fatura kesebileceği gibi sorular doğrudan şirketin finansal güvenliğini ilgilendirir. Authentication ile Authorization’ı karıştırmak — örneğin “kullanıcı giriş yaptı, o zaman her şeyi yapabilir” varsayımı — ciddi güvenlik açıklarına yol açar.

MiniCloudERP’de Nerede Kullanılır

- Basit “Admin/Satış Temsilcisi” ayrımı için Role-Based Authorization yeterlidir.
- “Bir satış temsilcisi sadece kendi oluşturduğu siparişi görebilir” gibi ince taneli bir kural gerektiğinde Claim-Based Authorization ve Policy’ler kullanılır.
- Şifreler asla düz metin saklanmaz; Identity, şifreleri hashleyerek saklar.

Kod Örneği

```
// Role-Based: sadece Admin rolündeki kullanıcılar erişebilir
[Authorize(Roles = "Admin")]
public class ProductManagementController : Controller { }

// Policy tanımı (Program.cs)
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanViewOwnOrdersOnly", policy =>
        policy.RequireClaim("Department", "Sales"));
});

// Claim-Based kullanım
[Authorize(Policy = "CanViewOwnOrdersOnly")]
public async Task<IActionResult> MyOrders() { /* ... */ }
```

Sık Yapılan Hatalar

- Authentication ile Authorization’ı karıştırmak: “giriş yaptı” ile “bu işlemi yapmaya yetkili” farklı sorulardır.
- Kullanıcı girdisini parametrelili sorgular yerine doğrudan SQL metnine eklemek; bu, SQL Injection açığına yol açar. EF Core kullanıldığında LINQ sorguları bu riski büyük ölçüde ortadan kaldırır, ancak elle yazılan ham SQL sorgularında dikkatli olunmalıdır.
- Kullanıcıdan gelen veriyi (örn. yorum, ürün açıklaması) hiç temizlemeden doğrudan sayfada göstermek; bu, XSS açığına yol açar. Blazor, varsayılan olarak HTML çıktısını otomatik kaçışlar (encode eder) ancak MarkupString gibi ham HTML gösteren yapılar dikkatli kullanılmalıdır.

Unutma: Basit bir “Admin/User” ayrımı yeterliyken gereksiz yere karmaşık claim/policy yapıları kurmak zaman kaybıdır — Claim-Based Authorization, yetki ihtiyacı basit bir rol grubuyla değil daha ince taneli/dinamik koşullarla ifade edilmesi gerektiğinde tercih edilir. Şifreler EF Core veya başka bir yolla asla düz metin olarak saklanmaz; Identity bunun için hashleme kullanır.

Özet

Identity ve güvenlik, bir ERP’nin “kim, neyi, ne zaman yapabilir” sorusuna cevap verir. Authentication/Authorization ayrımı, Role/Claim-Based yaklaşımlar ve temel

güvenlik açıkları (SQL Injection, XSS), MiniCloudERP gibi gerçek iş verisiyle çalışan bir sistemde göz ardı edilemeyecek konulardır.

BÖLÜM 11 — KATMANLI MİMARİ (DETAYLI)

Tanım

Katmanlı mimari (Layered / N-Tier Architecture), bir uygulamayı sorumluluk bazında yatay katmanlara ayırır. MiniCloudERP’de dört ana katman bulunur: **Presentation**, **Business**, **Repository** ve **Data**. Her katman yalnızca kendi altındaki katmanla konuşur; üst katmanlar alt katmanların iç detaylarını bilmez.

Katman	Sorumluluk	MiniCloudERP’deki Karşılığı
Presentation	Kullanıcı etkileşimi, ekran mantığı	Blazor bileşenleri (. razor)
Business (Service)	İş kuralları, doğrulama, iş akışı yönetimi	OrderService, StockService, InvoiceService
Repository	Veri erişimini soyutlama	IOrderRepository, IProductRepository
Data	Kalıcı veri saklama	AppDbContext (EF Core) + SQL Server

Terim	Açıklama
Separation of Concerns	Her katmanın/bileşenin yalnızca bir sorumluluğa odaklanması ilkesi
Repository Pattern	Veri erişim mantığını, iş mantığından soyutlayan tasarım deseni
Service Layer	İş kurallarını barındıran, Repository ile Presentation arasındaki katman
DTO (Data Transfer Object)	Katmanlar arasında veri taşımak için kullanılan, iş mantığı içermeyen basit veri nesnesi
Anemic Domain Model	İş mantığı barındırmayan, sadece veri tutan (get/set) entity yaklaşımı

Neden Önemli

Katmanlı mimari, bir değişikliğin etkisini sınırlar. Örneğin veritabanı SQL Server’dan başka bir sisteme taşınacak olsa, yalnızca Repository/Data katmanı değişir; Business ve Presentation katmanları etkilenmez — çünkü Business katmanı somut veritabanına değil, IOrderRepository gibi soyutlamalara bağımlıdır (bkz. Bölüm 5, Dependency Inversion).

MiniCloudERP’de Nerede Kullanılır

Bir sipariş oluşturulduğunda katmanlar şu sırayla devreye girer:

Presentation (Blazor)

|

v

Business (OrderService) -- iş kuralı: stok yeterli mi, müşteri aktif mi

|

v

Repository (OrderRepository, ProductRepository)

|

v

Data (AppDbContext + Transaction)

|

v

SQL Server

Kod Örneği

```
// Presentation katmanı: sadece kullanıcı etkileşimini yönetir
private async Task OnCreateOrderClicked()
{
    var result = await OrderService.CreateOrderAsync(newOrder);
    if (!result.IsSuccess)
    {
        errorMessage = result.ErrorMessage;
    }
}

// Business katmanı: iş kuralını uygular
public class OrderService : IOrderService
{
    public async Task<OrderResult> CreateOrderAsync(Order order)
    {
        var customer = await _customerRepository.GetByIdAsync(order.CustomerId);
        if (customer is null || !customer.IsActive)
            return OrderResult.Fail("Müşteri aktif değil.");

        foreach (var item in order.Items)
        {
            var hasStock = await _stockService.HasSufficientStockAsync(item.ProductId,
            if (!hasStock)
                return OrderResult.Fail($"Ürün {item.ProductId} için yeterli stok yok.");
        }

        await _orderRepository.AddAsync(order);
        return OrderResult.Success(order);
    }
}
```

Sık Yapılan Hatalar

- Bu katmanlardan birini (genellikle Business katmanını) atlayarak Blazor bileşeninden doğrudan Repository'yi çağırmak; bu, iş kurallarının UI kodu içine dağılmasına ve test edilemez hale gelmesine yol açar.
- Entity sınıflarını (örn. Order) doğrudan Presentation katmanına taşımak yerine DTO kullanmamak; bu, katmanlar arası gereksiz bağımlılık yaratabilir ve iç veri yapısının dışarı sızmasına yol açar.
- İş mantığını Service katmanı yerine entity içine (Anemic Domain Model karşıtı aşırı uçta) ya da hiç bir yere koymayıp Presentation'a dağıtmak.

Unutma: Bu katmanlardan birini “hızlı teslim etmek” için atlamak kısa vadede hızlı, uzun vadede yavaşlatıcı bir yaklaşımdır. Her şey tek bir dev sınıfta yazılmamalıdır çünkü bu Single Responsibility ve High Cohesion ilkelerini ihlal eder; kod zamanla anlaşılmaz, test edilemez ve değiştirilmesi riskli hale gelir.

Özet

Katmanlı mimari, MiniCloudERP'nin her bir parçasının kendi sorumluluğuna odaklanmasını ve değişikliklerin etkisinin sınırlı kalmasını sağlar. Presentation, Business, Repository ve Data katmanlarının her biri, önceki bölümlerde görülen SOLID ilkelerinin (özellikle Dependency Inversion) somut bir uygulamasıdır.

BÖLÜM 12 – GİT

Tanım

Git, kaynak kodundaki değişiklikleri zaman içinde takip eden, birden fazla geliştiricinin aynı kod tabanı üzerinde eşzamanlı çalışmasını mümkün kılan bir dağıtık sürüm kontrol sistemidir.

Terim	Açıklama
Repository (Depo)	Bir projenin tüm dosyalarının ve değişiklik geçmişinin tutulduğu yer
Commit	Belirli bir andaki değişikliklerin kalıcı bir kayıt olarak geçmişe eklenmesi
Branch (Dal)	Ana koddan bağımsız olarak geliştirme yapılabilen paralel bir çizgi
Merge (Birleştirme)	İki dalın değişikliklerini tek bir dala birleştirme işlemi
Pull Request / Merge Request	Bir dalın değişikliklerinin ana dala birleştirilmesi için açılan, inceleme (code review) sürecini başlatan istek
Conflict (Çakışma)	Aynı dosyanın aynı satırında iki farklı dalda yapılan değişikliklerin otomatik birleştirilemediği durum
Rebase	Bir dalın geçmişini, başka bir dalın en güncel haline göre yeniden yazma işlemi
Git Flow	Feature/Develop/Main gibi belirli dal isimlendirme ve birleştirme kurallarını tanımlayan yaygın bir çalışma modeli

Neden Önemli

MiniCloudERP gibi birden fazla geliştiricinin çalıştığı bir projede, Git olmadan iki kişinin aynı dosyada yaptığı değişiklikleri güvenli şekilde birleştirmek neredeyse imkansızdır. Doğru branch stratejisi ve düzenli, anlamlı commit'ler, hem ekip içi işbirliğini hem de bir hatanın ne zaman ve neden girdiğinin izlenebilirliğini sağlar.

MiniCloudERP’de Nerede Kullanılır

Bir geliştirici yeni bir özellik (örn. “toplu fatura oluşturma”) üzerinde çalışırken main dalından ayrılan bir feature/bulk-invoice dalı açar. Değişiklikler tamamlandığında bir Pull Request açılır; en az bir başka geliştirici kodu inceler (code review), onaylanırsa main dalına birleştirilir.

```
main -----o-----o-----> (canlıya alınan, kararlı kod)
              \               /
                o---o---o---o----- feature/bulk-invoice dalı
```

Kod Örneği

```
# Yeni bir özellik dalı oluşturma
git checkout -b feature/bulk-invoice

# Değişiklikleri kaydetme
git add .
git commit -m "Add bulk invoice generation service"

# Uzak depoya gönderme ve Pull Request açma
git push origin feature/bulk-invoice

# main dalındaki güncel değişiklikleri kendi dalına almak
git checkout feature/bulk-invoice
git pull origin main
```

Sık Yapılan Hatalar

- Tek bir commit içine ilgisiz birçok değişikliği (örn. hem yeni özellik hem de tamamen alakasız bir bug fix) sıkıştırmak; bu, ileride belirli bir değişikliği geri almayı (revert) veya izlemeyi zorlaştırır.
- Anlamsız commit mesajları yazmak (“fix”, “update”, “asdf”); commit geçmişi, ileride bir hatanın hangi değişiklikle geldiğini bulmak için önemli bir araçtır.
- main dalına doğrudan, code review olmadan kod göndermek (push); bu, kalite kontrolünü ve bilgi paylaşımını atlar.
- Bir çakışmayı (conflict) anlamadan, “her ikisini de” ya da rastgele bir tarafı seçerek çözmek; bu, sessizce hatalı kodun birleşmesine yol açabilir.

Unutma: Pull Request, sadece kodu birleştirme aracı değil, aynı zamanda ekip içi bilgi paylaşımının ve kalite kontrolünün de aracıdır — kod incelemesi (code review) burada gerçekleşir. Bir çakışma (conflict) oluştuğunda, hangi değişikliğin doğru olduğunu anlamadan otomatik bir seçim yapmak, sessizce hatalı kodun birleşmesine yol açabilir.

Özet

Git, MiniCloudERP gibi çok geliştiricili bir projede kodun güvenli, izlenebilir ve geri alınabilir şekilde geliştirilmesini sağlar. Branch stratejisi, anlamlı commit’ler ve Pull Request süreci, teknik beceri kadar ekip çalışması disiplini de yansıtır.

BÖLÜM 13 — TEST YAZMA

Tanım

Test yazma, kodun beklenen şekilde çalıştığını otomatik olarak doğrulayan, tekrar tekrar çalıştırılabilen kod parçaları oluşturma pratiğidir. ERP gibi doğrudan finansal sonuçları olan sistemlerde test, “çalışıyor gibi görünüyor” ile “doğru çalıştığı kanıtlanmış” arasındaki farkı oluşturur.

Terim	Açıklama
Unit Test (Birim Testi)	Tek bir sınıf veya metodu, dış bağımlılıklardan izole ederek test etme
Integration Test (Entegrasyon Testi)	Birden fazla bileşenin (örn. Service + gerçek veya in-memory veritabanı) birlikte doğru çalıştığını test etme
Mock (Sahte Nesne)	Gerçek bir bağımlılığın (örn. veritabanı, dış servis) yerine geçen, test sırasında kontrollü davranış sergileyen sahte nesne
Arrange-Act-Assert (AAA)	Bir testi üç net bölüme ayıran yaygın yapı: hazırlık, işlemi çalıştırma, sonucu doğrulama
Test Coverage (Test Kapsamı)	Kodun ne kadarının testler tarafından çalıştırıldığını gösteren metrik
Regression (Gerileme)	Önceden doğru çalışan bir özelliğin, yeni bir kod değişikliğiyle bozulması
TDD (Test-Driven Development)	Önce testi, sonra testi geçecek kodu yazma geliştirme yaklaşımı

Neden Önemli

Bir ERP’de stok hesaplama, indirim mantığı veya yetkilendirme kuralındaki bir hata, gerçek parasal kayıplara yol açabilir. Otomatik testler, bu tür kritik iş kurallarının her kod değişikliğinden sonra hâlâ doğru çalıştığını hızlıca doğrulamayı sağlar — manuel olarak her seneryoyu elle test etmek hem yavaş hem de hataya açıktır.

MiniCloudERP’de Nerede Kullanılır

Stok kontrolü, sipariş durum geçişleri, indirim/fiyat hesaplama ve yetkilendirme kontrolleri gibi doğrudan parasal veya iş sonucu etkileyen kurallar mutlaka test edilir. Bir sipariş servisi test edilirken gerçek veritabanı yerine `IOrderRepository`’nin sahte (mock) bir implementasyonu kullanılır — böylece test, yavaş ve kararsız dış bağımlılıklardan izole, hızlı ve tekrarlanabilir şekilde çalışır.

Kod Örneği

```
public class OrderServiceTests
{
    [Fact]
    public async Task CreateOrderAsync_YetersizStok_SiparisiReddetmeli()
    {
        // Arrange (hazırlık)
        var mockStockService = new Mock<IStockService>();
        mockStockService.Setup(s => s.HasSufficientStockAsync(It.IsAny<int>()), It.IsAny<Task>().ReturnsAsync(false));
        var mockRepository = new Mock<IOrderRepository>();
        var sut = new OrderService(mockRepository.Object, mockStockService.Object);
        var order = new Order { Items = { new OrderItem { ProductId = 1, Quantity = 10 } } };

        // Act (işlemi çalıştırma)
        var result = await sut.CreateOrderAsync(order);

        // Assert (sonucu doğrulama)
        Assert.False(result.IsSuccess);
        mockRepository.Verify(r => r.AddAsync(It.IsAny<Order>()), Times.Never);
    }
}
```

Sık Yapılan Hatalar

- Sadece basit CRUD işlemlerini test edip, kritik iş kurallarını (edge case'leri) hiç test etmemek.
- Mock'ları o kadar karmaşık hale getirmek ki testin kendisi, test edilen gerçek mantıktan daha karmaşık hale gelir.
- Sadece Test Coverage yüzdesini yükseltmek için anlamsız, hiçbir gerçek senaryoyu doğrulamayan testler yazmak — %100 kapsama oranı, testlerin anlamlı olduğu anlamına gelmez.
- Yeni bir özellik eklerken, ilgisiz görünen ama aslında bağlı olan eski bir özelliği (regression riski) test etmeden bırakmak.

Unutma: Integration Test'te bazen gerçek SQL Server yerine in-memory veritabanı kullanılır — bu, hızlı ve izole bir ortam sağlar; ancak in-memory veritabanı, gerçek SQL Server'ın bazı davranışlarını (belirli kısıtlamalar, SQL'e özgü fonksiyonlar) birebir yansıtmayabilir, bu yüzden production davranışının %100 aynı olduğu varsayılmamalıdır. Debugging sırasında breakpoint, kodun belirli bir satırda durmasını sağlayarak o anki değişken değerlerini ve akışı incelemeye imkan tanır; sadece Console.WriteLine eklemek yerine debugger'ı aktif kullanmak önemli bir alışkanlıktır.

Özet

Test yazma, MiniCloudERP'nin kritik iş kurallarının her değişiklikten sonra doğru çalışmaya devam ettiğini garanti eden bir güvenlik ağıdır. Unit Test, Integration Test ve düzenli code review alışkanlığı birlikte, uzun ömürlü bir ürünün kalitesini korur.

BÖLÜM 14 — MINICLOUDERP’DE UÇTAN UCA AKIŞ

Bu bölüm, önceki bölümlerde ayrı ayrı işlenen kavramların gerçek bir senaryoda nasıl bir araya geldiğini gösterir: bir müşterinin sipariş vermesinden faturanın oluşmasına kadar geçen tüm akış.

Senaryo: Bir Siparişin Baştan Sona Yolculuğu

1. Kullanıcı arayüzde siparişi oluşturur (Bölüm 7 — Blazor Server) Kullanıcı “Yeni Sipariş” ekranında ürünleri seçip “Siparişi Oluştur” butonuna tıklar. Bu tıklama SignalR üzerinden sunucudaki bileşen örneğine iletilir.

2. İstek Business katmanına gider (Bölüm 11 — Katmanlı Mimari) Blazor bileşeni, doğrudan veritabanına dokunmaz; IOrderService arayüzü üzerinden OrderService’i çağırır (Bölüm 5 — Dependency Inversion).

3. İş kuralları uygulanır OrderService, müşterinin aktif olup olmadığını kontrol eder, her ürün için stok yeterliliğini StockService üzerinden sorgular (Bölüm 1 — ERP İş Kuralları).

4. Veri katmanına ulaşılır (Bölüm 8 — Entity Framework Core) Stok yeterliyse, sipariş kaydı ve stok güncellemesi bir **Transaction** içinde birlikte veritabanına yazılır — böylece “stok düştü ama sipariş kaydedilmedi” gibi bir tutarsızlık oluşmaz.

5. Veritabanı işlemi gerçekleşir (Bölüm 9 — SQL Server) Orders ve OrderItems tabloları, foreign key ilişkileriyle güncellenir; CustomerId üzerindeki index sayesinde bu müşterinin geçmiş siparişleri de hızlıca sorgulanabilir.

6. Yetkilendirme her adımda geçerlidir (Bölüm 10 — Identity ve Güvenlik) Kullanıcının bu siparişi oluşturmaya yetkili olup olmadığı, [Authorize] ve ilgili Policy kontrolleriyle daha istek Business katmanına ulaşmadan doğrulanmıştır.

7. Sonuç arayüze geri döner (Bölüm 7 — Blazor Server) İşlem tamamlandığında, yalnızca değişen HTML parçası (örn. yeni sipariş satırı) tarayıcıya gönderilir; tüm sayfa yeniden yüklenmez.

8. Değişiklik Git üzerinden ekibe ulaşır (Bölüm 12 — Git) Bu akışı oluşturan kod, bir feature dalında geliştirilmiş, Pull Request ile incelenmiş ve main dalına birleştirilmiştir.

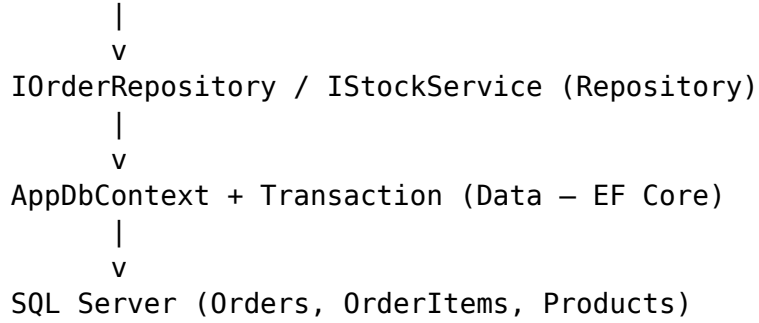
9. Testler, akışın doğruluğunu garanti eder (Bölüm 13 — Test Yazma) “Yetersiz stok varsa sipariş reddedilmeli” gibi kritik senaryolar, otomatik testlerle her kod değişikliğinden sonra tekrar doğrulanır.

10. Sistem, bulutta ölçeklenir (Bölüm 2 — Bulut Bilişim) Yoğun dönemlerde (örn. kampanya günü) bu akış, Load Balancer arkasında birden fazla uygulama sunucusunda paralel olarak çalışabilir; sık sorgulanan “günlük toplam satış” gibi değerler Redis’te önbelleklenir.

Kullanıcı (Blazor UI)



OrderService (Business – iş kuralları)



Bu Akıştan Çıkarılacak Ders

Hiçbir katman veya kavram tek başına yeterli değildir. ERP terminolojisi *neyin* yapıldığını, C#/OOP/Katmanlı Mimari *nasıl* yapıldığını, Cloud ve Güvenlik ise *nerede* ve *ne kadar güvenli* yapıldığını açıklar. Bir Junior Developer olarak bu bağlantıyı — bir iş kuralının nasıl kod haline geldiğini, kodun nasıl katmanlardan geçtiğini, verinin nasıl kalıcı hale geldiğini — görebilmek, tek tek kavramları ezberlemekten çok daha değerlidir.

EK A — HIZLI MÜLAKAT REFERANSI

Bu ek, önceki bölümlerin tam metnini tekrar etmez; her konu için mülakatta sıkça sorulan sorulara **tek cümlelik, doğrudan** cevaplar sunar. Amaç, mülakattan 1-2 gün önce hızlıca göz gezdirip hafızayı tazelemektir. Detaylı açıklama ve örnekler için ilgili bölüme dönmelidir.

ERP ve Bulut

- **ERP nedir?** Bir şirketin temel iş süreçlerini tek bir merkezi sistemde birleştiren yazılım türü.
- **Cloud ERP ile On-Premise farkı nedir?** Cloud ERP bulut sağlayıcıda çalışır ve internetten erişilir; On-Premise şirketin kendi sunucusunda kurulur ve bakımı şirkete aittir.
- **Load Balancer ne işe yarar?** Gelen istekleri birden fazla sunucuya dağıtarak yükü dengeler ve bir sunucu çöktüğünde hizmetin devam etmesini sağlar.
- **Container ile Virtual Machine farkı nedir?** Container işletim sistemini paylaşır ve hafiftir; Virtual Machine kendi işletim sistemini taşır ve daha ağırdır.

C# ve OOP

- **Task ile Task<T> farkı nedir?** Task değer döndürmeyen, Task<T> ise T tipinde değer döndüren asenkron işlemi temsil eder.
- **SOLID'deki "D" neden önemlidir?** İş mantığının somut detaylara değil soyutlamalara (interface) bağımlı olmasını sağlar, böylece test edilebilirlik ve esneklik artar.
- **DRY ile YAGNI çelişir mi?** Hayır; DRY tekrarı azaltmayı, YAGNI henüz gerekmeyen özelliği eklememeyi savunur — ikisi birlikte "gerekeni tekrarsız yaz" ilkesini oluşturur.
- **Interface neden kullanılır?** Sınıfın "ne yapabildiğini" tanımlar, "nasıl yaptığını" gizler; farklı implementasyonların birbirinin yerine kullanılmasını sağlar.

ASP.NET Core ve Blazor

- **Dependency Injection nedir?** Bir sınıfın ihtiyaç duyduğu bağımlılıkların dışarıdan (genellikle constructor ile) sağlanması.
- **Scoped neden DbContext için doğru seçimdir?** DbContext thread-safe değildir; Scoped, aynı istek/devre içinde tek bir örnek paylaşılmasını garanti eder.
- **Blazor Server'da bir tıklama nasıl işlenir?** SignalR üzerinden sunucudaki bileşene iletilir, ilgili metot çalışır, sadece değişen HTML parçası tarayıcıya gönderilir.
- **Authentication ile Authorization farkı nedir?** Authentication "kim olduğunu", Authorization "neyi yapmaya yetkili olduğunu" doğrular.

Entity Framework Core ve SQL Server

- **N+1 problemi nedir, nasıl önlenir?** Bir liste sorgusundan sonra her öge için ayrı sorgu atılmasıdır; Include (eager loading) ile önlenir.
- **Transaction neden gereklidir?** Birden fazla tabloyu etkileyen bir işlemin ya tamamen başarılı ya da hiç uygulanmamış olmasını garanti eder.
- **Index'in maliyeti nedir?** Okuma hızını artırır ama her yazma işleminde (INSERT/UPDATE) ek güncelleme maliyeti getirir.
- **ACID neyi ifade eder?** Atomicity, Consistency, Isolation, Durability — bir veritabanı işleminin güvenilirlik garantileri.

Identity, Mimari, Git ve Test

- **Claim-Based Authorization ne zaman tercih edilir?** Yetki ihtiyacı basit bir rolle değil, ince taneli/dinamik koşullarla ifade edilmesi gerektiğinde.
- **Katmanlı mimaride bir katman neden atlanmamalı?** Atlamak, iş kurallarının yanlış katmana (örn. UI koduna) dağılmasına ve test edilemez hale gelmesine yol açar.
- **Mock nedir?** Gerçek bir bağımlılığın yerine geçen, test sırasında kontrollü davranan sahte nesne.
- **Pull Request'in amacı nedir?** Kod değişikliğinin ana dala birleşmeden önce incelenmesini (code review) ve kalite kontrolünü sağlamak.
- **Teknik borç nedir?** Kısa vadeli hız kazanmak için alınan, uzun vadede ek bakım maliyeti getiren tasarım/kod kararı.

Genel Davranış Soruları

- **Bir gereksinimi tam anlamadığında ne yapılmalı?** Varsayımda bulunmak yerine ilgili kişiye (senior developer, iş analisti) soru sorulmalı.
- **İlk aylarda bir junior geliştirici neye odaklanmalı?** Mevcut kod tabanını anlamaya, katman sorumluluklarına saygı duymaya, küçük ve test edilmiş değişiklikler yapmaya, soru sormaktan çekinmemeye.
- **"Neden bu teknolojiyi/deseni kullandın?" sorusuna nasıl cevap verilmeli?** Kararın hangi problemi çözdüğü (test edilebilirlik, gevşek bağımlılık, performans) açıkça ifade edilerek — sadece "böyle öğrendim" değil.

KAPANIŞ

Bu kitap, MiniCloudERP örneği üzerinden bir Cloud ERP uygulamasının temelden büyük resme kadar tüm katmanlarını kapsadı: ERP iş kavramları, bulut altyapısı, C# ve OOP temelleri, ASP.NET Core ve Blazor Server ile uygulama katmanı, Entity Framework Core ve SQL Server ile veri katmanı, Identity ile güvenlik, katmanlı mimari, Git ve test yazma alışkanlıkları ve son olarak tüm bunların uçtan uca nasıl bir araya geldiği.

Unutma: Bu bir kere okunup bırakılan bir doküman değildir. Yeni bir kavramla karşılaşıldığında, özellikle bir mülakat öncesinde, ilgili bölüme geri dönmelidir.

Başarılar dileriz.